YAMAGoya
Open Source Threat Hunting Tool using YARA and SIGMA

Shusei Tomonaga
Tomoya Kamei

CODE BLUE 2025
BECAUSE SECURITY MATTERS

Motivation

Endpoint protection products exist for malware detection, targeted attacks utilizing malware that operates only in memory remain challenging to detect using existing products and continue to pose a threat.

Endpoint protection to detect non-file-based activity depends on the black box of the security solution and cannot detect activities using legitimate tools used by the attackers.

The best way to threat hunting is to use YARA and SIGMA, and we need a security solution that uses those rules to endpoint detection.

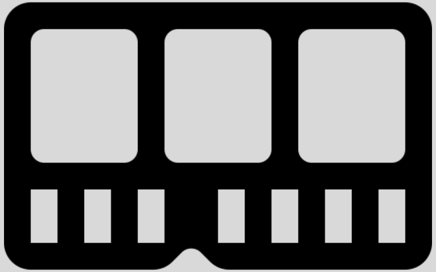

We have created a new tool.

New Tool Concept

Create a threat hunting tool that can use **YARA** and **SIGMA**, knowledge of the security community.

Targets of This Tool

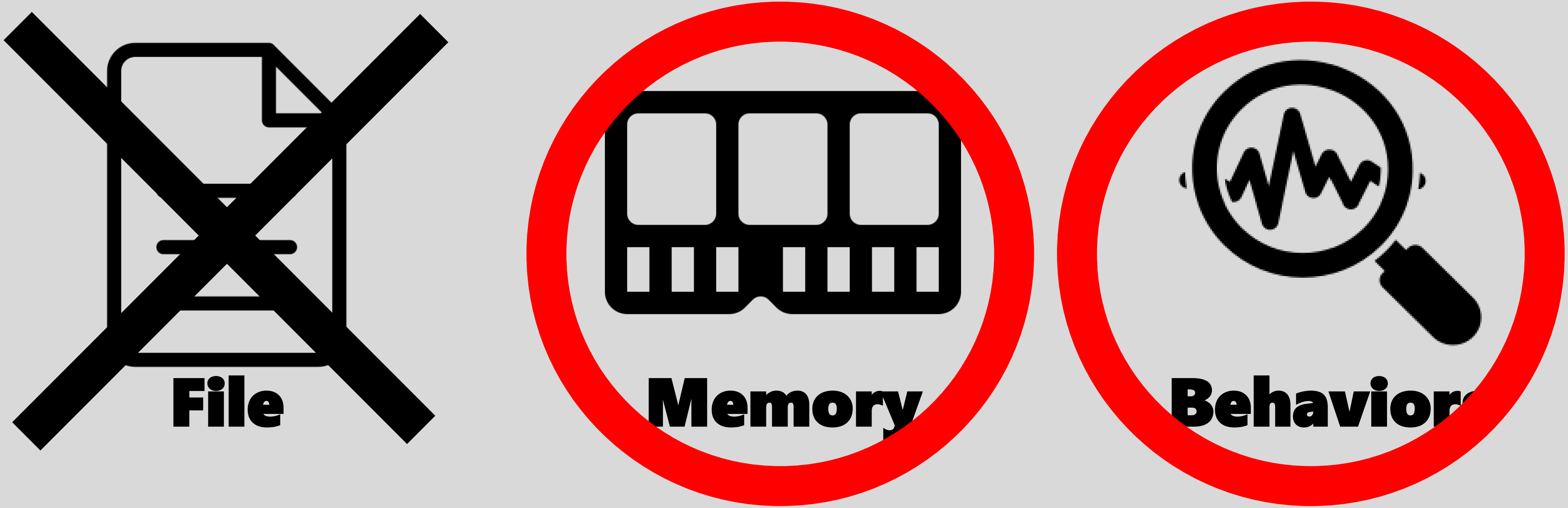
File



Memory

Behaviors

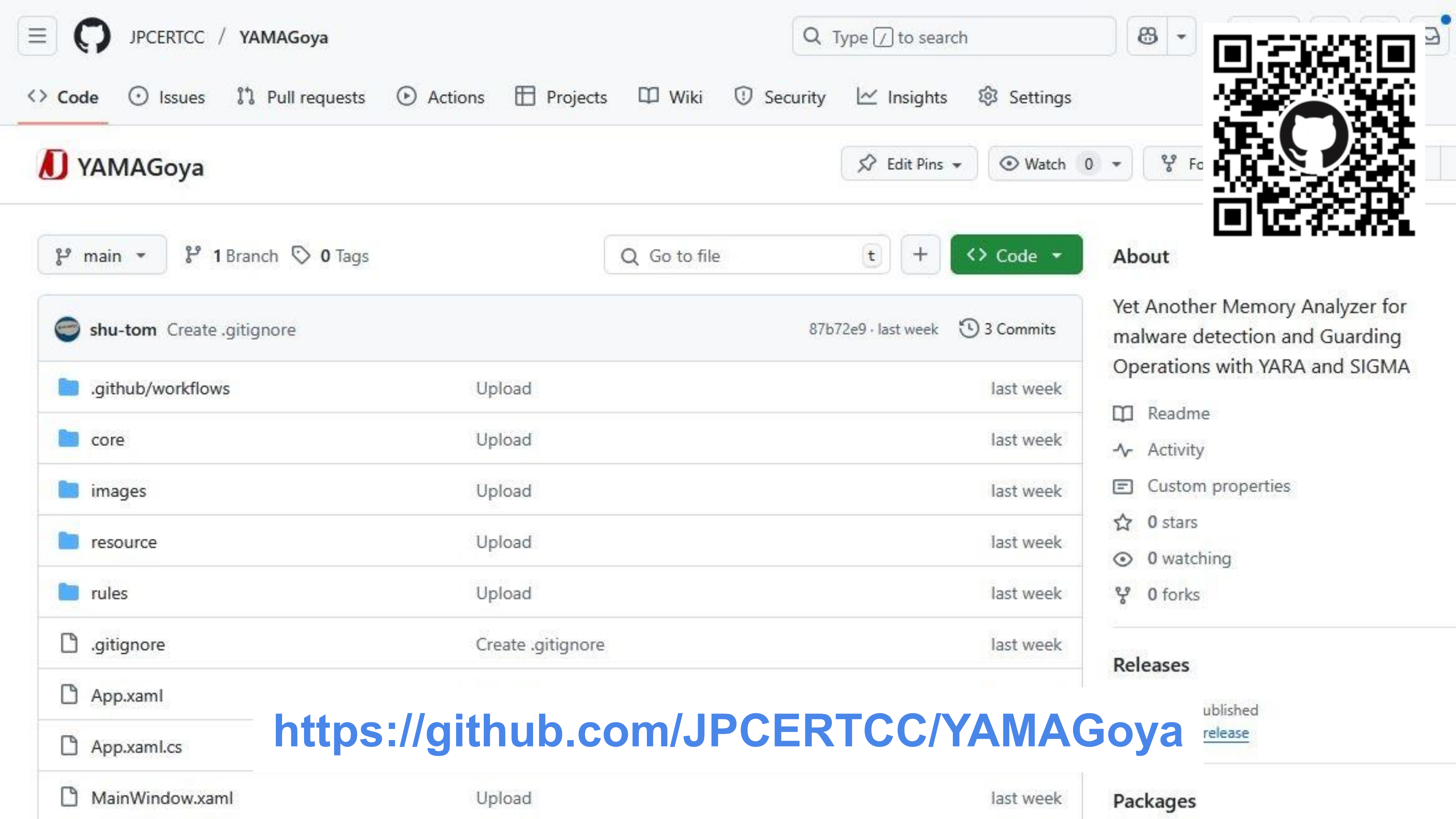
Targets of This Tool



This tool is not a file scanner.

What is YAMAGoya

YAMAGoya is a C# application that leverages Event Tracing for Windows (ETW) to capture real-time system events and memory during threat hunting or incident response.



https://github.com/JPCERTCC/YAMAGoya

What is YAMAGoya

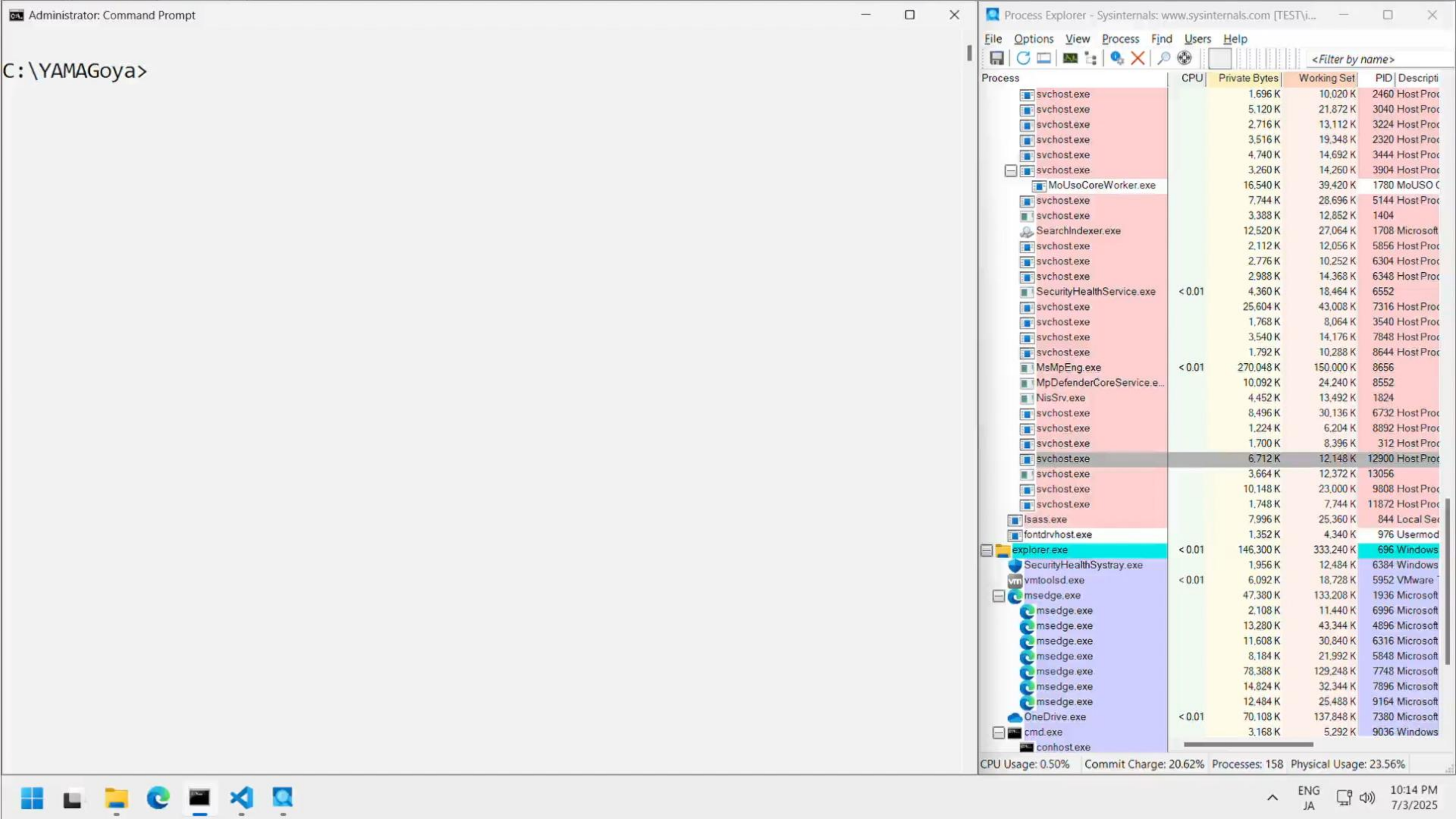
YAMAGoya

(Yet Another Memory Analyzer for malware detection and Guarding Operations with YARA and SIGMA)

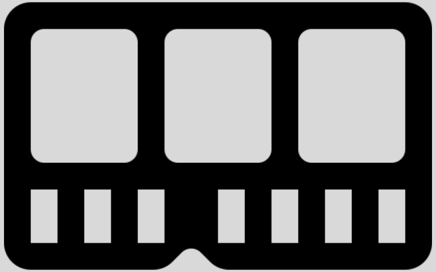
Features

1. Behavioral detection using **SIGMA** rules.
2. Detects malware using **YARA** rules and **memory information.**
3. Execute entirely in **userland**, avoiding kernel-mode dependencies.
4. Support for **GUI** and **CUI.**

Demo
- Malware detection using YAMAGoya -


Administrator: Command Prompt
C:\YAMAGoya>
Process Explorer - Sysinternals: www.sysinternals.com [TEST\i...
File Options View Process Find Users Help
<Filter by name>
Process CPU Private Bytes Working Set PID Descripti
svchost.exe 1,696 K 10,020 K 2460 Host Proc
svchost.exe 5,120 K 21,872 K 3040 Host Proc
svchost.exe 2,716 K 13,112 K 3224 Host Proc
svchost.exe 3,516 K 19,348 K 2320 Host Proc
svchost.exe 4,740 K 14,692 K 3444 Host Proc
svchost.exe 3,260 K 14,260 K 3904 Host Proc
MoUsoCoreWorker.exe 16,540 K 39,420 K 1780 MoUSO C
svchost.exe 7,744 K 28,696 K 5144 Host Proc
svchost.exe 3,388 K 12,852 K 1404
SearchIndexer.exe 12,520 K 27,064 K 1708 Microsoft
svchost.exe 2,112 K 12,056 K 5856 Host Proc
svchost.exe 2,776 K 10,252 K 6304 Host Proc
svchost.exe 2,988 K 14,368 K 6348 Host Proc
SecurityHealthService.exe < 0.01 4,360 K 18,464 K 6552
svchost.exe 25,604 K 43,008 K 7316 Host Proc
svchost.exe 1,768 K 8,064 K 3540 Host Proc
svchost.exe 3,540 K 14,176 K 7848 Host Proc
svchost.exe 1,792 K 10,288 K 8644 Host Proc
MsMpEng.exe < 0.01 270,048 K 150,000 K 8656
MpDefenderCoreService.e... 10,092 K 24,240 K 8552
NisSrv.exe 4,452 K 13,492 K 1824
svchost.exe 8,496 K 30,136 K 6732 Host Proc
svchost.exe 1,224 K 6,204 K 8892 Host Proc
svchost.exe 1,700 K 8,396 K 312 Host Proc
svchost.exe 6,712 K 12,148 K 12900 Host Proc
svchost.exe 3,664 K 12,372 K 13056
svchost.exe 10,148 K 23,000 K 9808 Host Proc
svchost.exe 1,748 K 7,744 K 11872 Host Proc
lsass.exe 7,996 K 25,360 K 844 Local Sec
fontdrvhost.exe 1,352 K 4,340 K 976 Usermod
explorer.exe < 0.01 146,300 K 333,240 K 696 Windows
SecurityHealthSystray.exe 1,956 K 12,484 K 6384 Windows
vmtoolsd.exe < 0.01 6,092 K 18,728 K 5952 VMware
msedge.exe 47,380 K 133,208 K 1936 Microsoft
msedge.exe 2,108 K 11,440 K 6996 Microsoft
msedge.exe 13,280 K 43,344 K 4896 Microsoft
msedge.exe 11,608 K 30,840 K 6316 Microsoft
msedge.exe 8,184 K 21,992 K 5848 Microsoft
msedge.exe 78,388 K 129,248 K 7748 Microsoft
msedge.exe 14,824 K 32,344 K 7896 Microsoft
msedge.exe 12,484 K 25,488 K 9164 Microsoft
OneDrive.exe < 0.01 70,108 K 137,848 K 7380 Microsoft
cmd.exe 3,168 K 5,292 K 9036 Windows
conhost.exe
CPU Usage: 0.50% Commit Charge: 20.62% Processes: 158 Physical Usage: 23.56%
ENG
JA
10:14 PM
7/3/2025

Targets of This Tool



Memory

Behaviors

How it Works

YAMAGoya operates as a Windows UserMode application, analyzing the memory spaces of processes running in user mode, scanning memory spaces with suspicious attributes using YARA, and identifying malware.

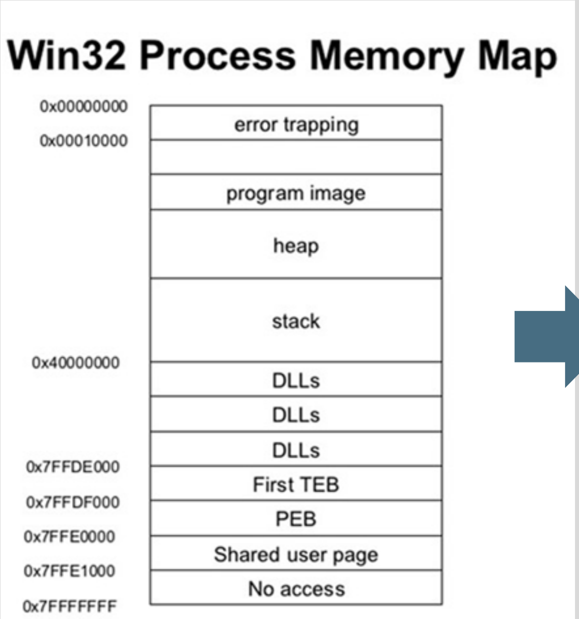
How it Works

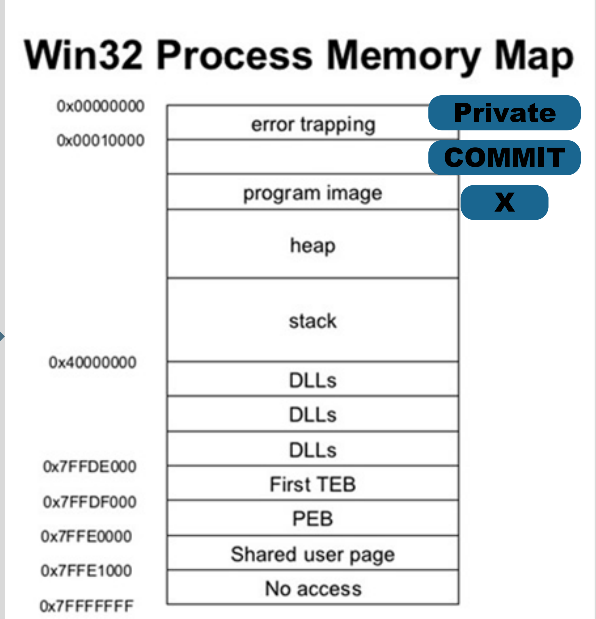
1. Analyzes each process and extracts the process information.
2. Determines the memory spaces to be inspected based on the analyzed information.
3. Inspects the targeted memory spaces using YARA rules.

Extract the Process information

1. Virtual Memory Map
2. Process PEB (Process Environment Block) structure
3. Process TEB (Thread Environment Block) structure
4. Process stack region
5. Process heap region
6. Process thread information

Tag the Memory Space to Scan





Scan with yara

Demo
- Memory scan using YARA rule -

ホーム | Debian12.6 | Windows11

Administrator: Command Prompt

```
C:\YAMAGoya>
C:\YAMAGoya>
```

rules

Lazarus_downloader_code.yara

Lazarus_downloader_code.yara

```
rule Lazarus_downloader_code {
   meta:
      description = "Lazarus downl
      author = "JPCERT/CC Incident
      hash = "faba4114ada285987d4

   strings:
      $jmp = { 53 31 c0 50 50 50 
      $count = { 00 00 EB 00 B8 FF
      $api = "InitOnceExecuteOnce"

   condition:
     uint16(0) == 0x5A4D and
     uint32(uint32(0x3c)) == 0x000
     filesize < 200KB and
     all of them
}
```

Ⓧ 0 ⚠ 0 Spaces: 3 UTF-8 CRLF {} Plain Text

Targets of This Tool

Memory

Behaviors

How it Works

YAMAGoya uses ETW event to collect various activities (file, registry, network, powershell, WMI etc.) within Windows OS and detect attack activities.

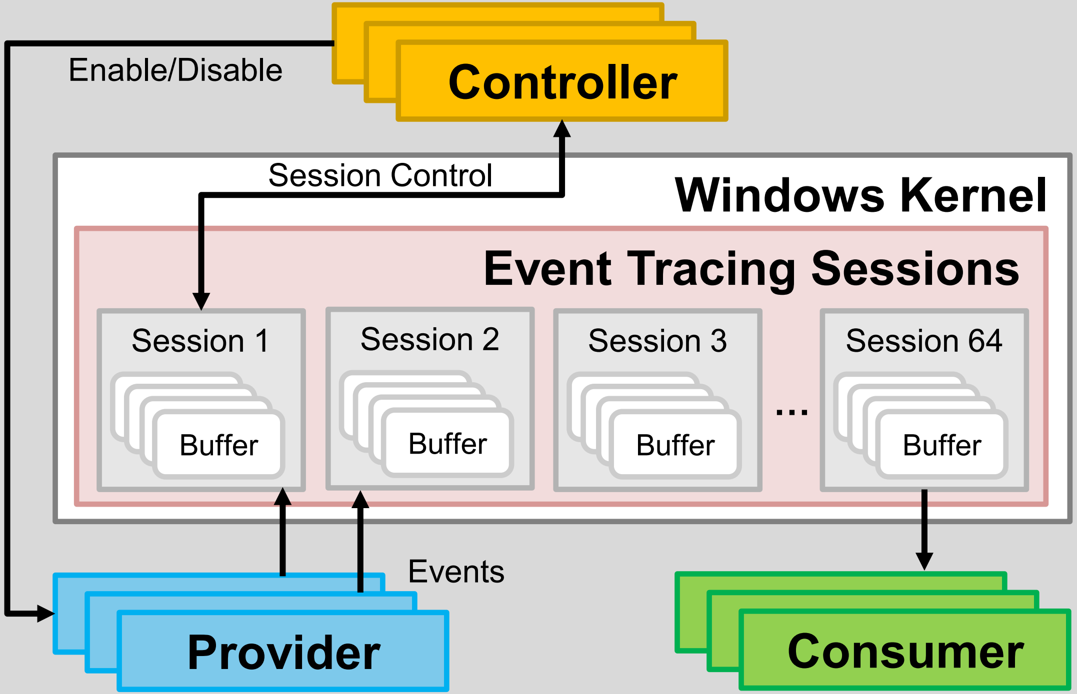
What is Event Tracing for Windows

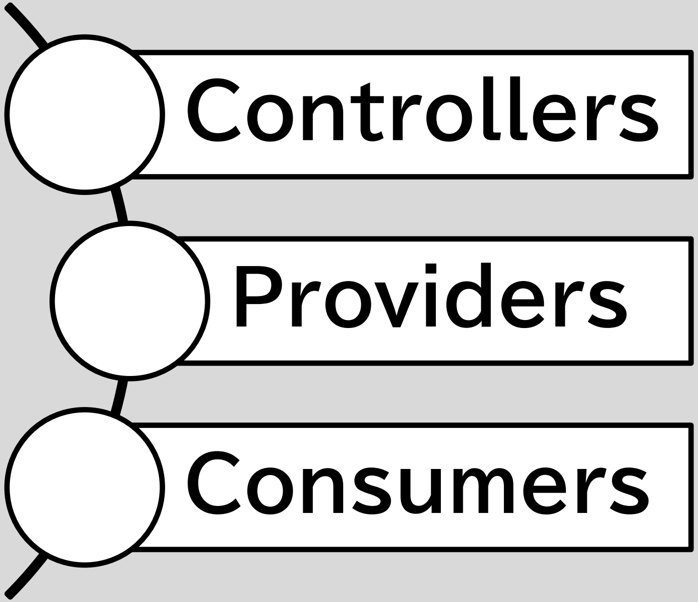
Event Tracing for Windows is an efficient kernel-level tracing facility that lets you log kernel or application-defined events to a log file. You can consume the events in real time or from a log file and use them to debug an application or to determine where performance issues are occurring in the application.

Use Case for ETW

- Application debugging
- EDR
- Event Log

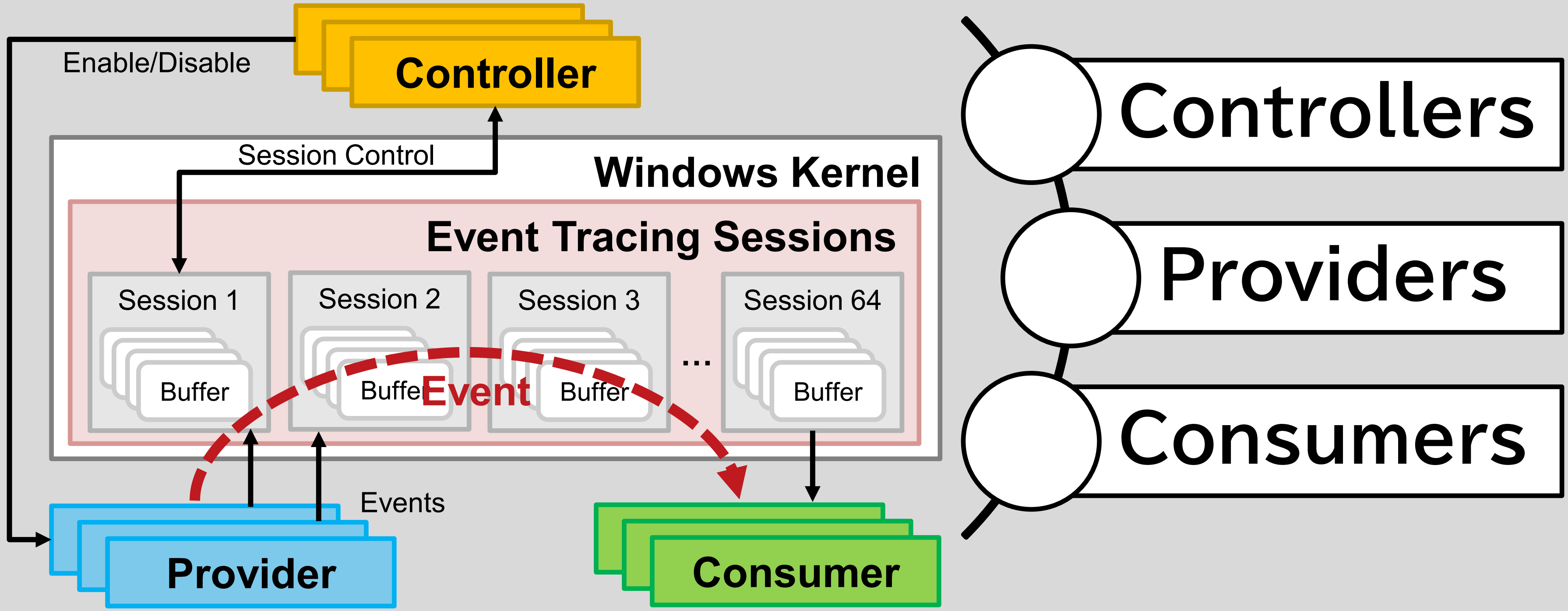
What is Event Tracing for Windows(ETW)





https://learn.microsoft.com/ja-jp/windows/win32/etw/about-event-tracing

What is Event Tracing for Windows(ETW)



https://learn.microsoft.com/ja-jp/windows/win32/etw/about-event-tracing

How it Works

1. Set up an ETW provider.
2. Collect **ETW events** to real-time.
3. Search for related events based on the **SIGMA rules** and other rulesets.

ETW Providers used by AV/EDR

Research AV/EDR

- Windows Defender
- Kaspersky
- ESET
- TrendMicro
- Symantec
- McAfee
- Cylance
- Filebeat (Elastic)

ETW Providers used by AV/EDR

Research AV/EDR

- Windows Defender
- Kaspersky
- ESET
- TrendMicro
- Symantec
- McAfee
- Cylance
- Filebeat (Elastic)

```
Microsoft-Windows-AppModel-Runtime
Microsoft-Windows-AppxPackagingOM
Microsoft-Windows-CAPI2
Microsoft-Windows-Crypto-NCrypt
Microsoft-Windows-Deplorch
Microsoft-Windows-DNS-Client
Microsoft-Windows-Eventlog
Microsoft-Windows-KnownFolders
Microsoft-Windows-LDAP-Client
Microsoft-Windows-NetworkProfile
Microsoft-Windows-Perflib
Microsoft-Windows-PrintService
Microsoft-Windows-RestartManager
Microsoft-Windows-RPC-Events
Microsoft-Windows-Shell-Core
Microsoft-Windows-User Profiles General
Microsoft-Windows-UserPnp
Microsoft-Windows-VerifyHardwareSecurity
Microsoft-Windows-WinHttp
Microsoft-Windows-WinINet-Pca
Network Location Awareness Trace
Network Profile Manager
WLAN Diagnostics Trace
```

ETW Providers used by AV/EDR

Research AV/EDR

- Windows Defender
- Kaspersky
- ESET
- TrendMicro
- Symantec
- McAfee
- Cylance →
- Filebeat (Elastic)

Microsoft-Windows-AppModel-Runtime
Microsoft-Windows-CAPI2
Microsoft-Windows-Crypto-NCrypt
Microsoft-Windows-Deplorch
Microsoft-Windows-DNS-Client
Microsoft-Windows-Eventlog
Microsoft-Windows-Kernel-AppCompat
Microsoft-Windows-KnownFolders
Microsoft-Windows-LDAP-Client
Microsoft-Windows-Networking-Correlation
Microsoft-Windows-RPC
Microsoft-Windows-RPC-Events
Microsoft-Windows-Shell-Core
Microsoft-Windows-User Profiles General
Microsoft-Windows-UserPnp
Microsoft-Windows-WinHttp
Microsoft-Windows-WinINet-Pca

ETW Providers used by AV/EDR

Important ETW Provider List

Microsoft-Windows-AppModel-Runtime	Microsoft-Windows-PrintService
Microsoft-Windows-CAPI2	Microsoft-Windows-RPC
Microsoft-Windows-COMRuntime	Microsoft-Windows-RPC-Events
Microsoft-Windows-Crypto-NCrypt	Microsoft-Windows-Shell-Core
Microsoft-Windows-Deplorch	Microsoft-Windows-User Profiles General
Microsoft-Windows-DNS-Client	Microsoft-Windows-UserPnp
Microsoft-Windows-Eventlog	Microsoft-Windows-VerifyHardwareSecurity
Microsoft-Windows-Kernel-AppCompat	Microsoft-Windows-WinHttp
Microsoft-Windows-KnownFolders	Microsoft-Windows-WinINet-Pca
Microsoft-Windows-LDAP-Client	Network Profile Manager
Microsoft-Windows-Networking-Correlation	WLAN Diagnostics Trace
Microsoft-Windows-NetworkProfile	

ETW Providers used by YAMAGoya

ETW Provider	ID	Details
Microsoft-Windows-Kernel-File	12,15	Create, Read
Microsoft-Windows-Kernel-Process	1	ProcessStart
Microsoft-Windows-Kernel-Registry	1,2,5,7	CreateKey, OpenKey, SetValueKey, QueryValueKey
Microsoft-Windows-DNS-Client	3000-3020	Query
Microsoft-Windows-Kernel-Network	-	TCP/IP
Microsoft-Windows-PowerShell	4104	StartingCommand
Microsoft-Windows-Shell-Core	9707, 28115	ExecutingFromRunKeyAsJobStart, AddShortcut
Microsoft-Windows-WinINet-Capture	-	WinINet
Microsoft-Windows-WinRM	6, 91	WSManSessioninitializeStart, WinRM connection
Microsoft-Windows-WMI-Activity	11	WMI connection

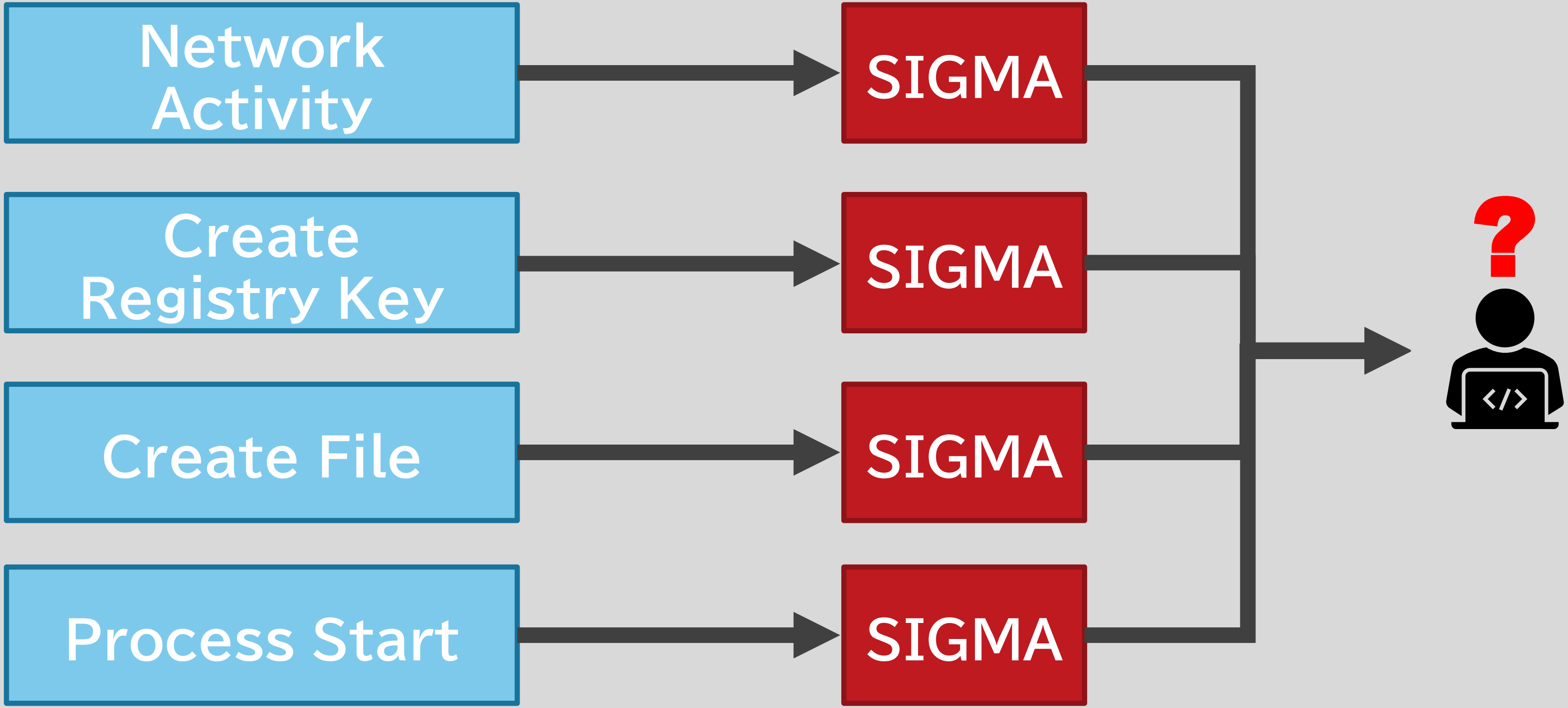
ETW Providers used by YAMAGoya

ETW Provider	ID	Details
Microsoft-Windows-Security-Mitigations	-	https://learn.microsoft.com/ja-jp/defender-endpoint/exploit-protection
Microsoft-Windows-Security-Adminless	-	Adminless mode
Microsoft-Windows-Security-LessPrivilegedAppContainer	-	
Microsoft-Windows-Audit-CVE	-	
Microsoft-Windows-SMBServer	552, 650, 700	Smb2SessionAuthenticated, Smb2FileOpenStart, Smb2ShareAdd
Microsoft-Windows-SMBClient	31010, 30501, 30508	Authorization,
Microsoft-Windows-Kernel-EventTracing	2, 3, 10, 11	ETW session start / stop

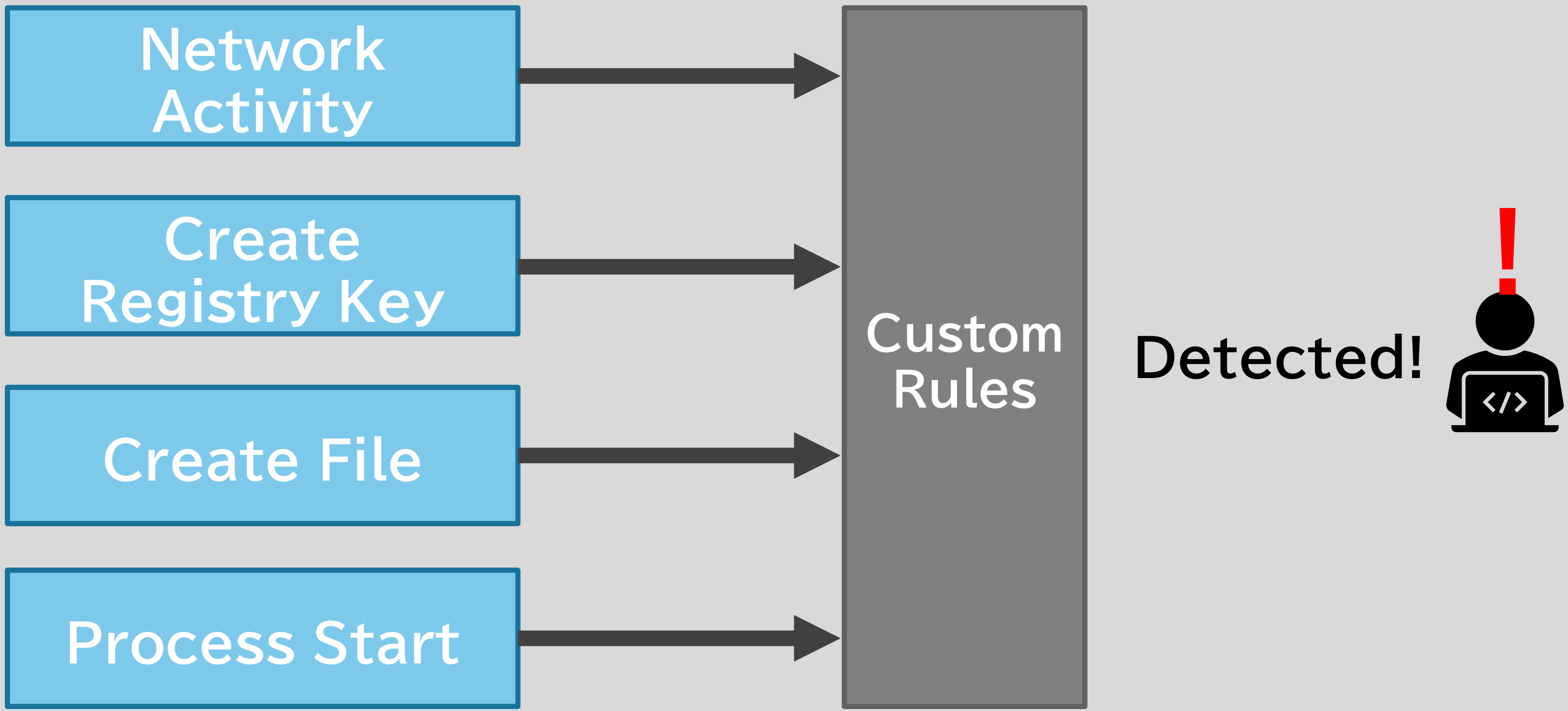
Advanced Attack Behavior Analysis

Network Activity	→ SIGMA
Create Registry Key	→ SIGMA
Create File	→ SIGMA
Process Start	→ SIGMA

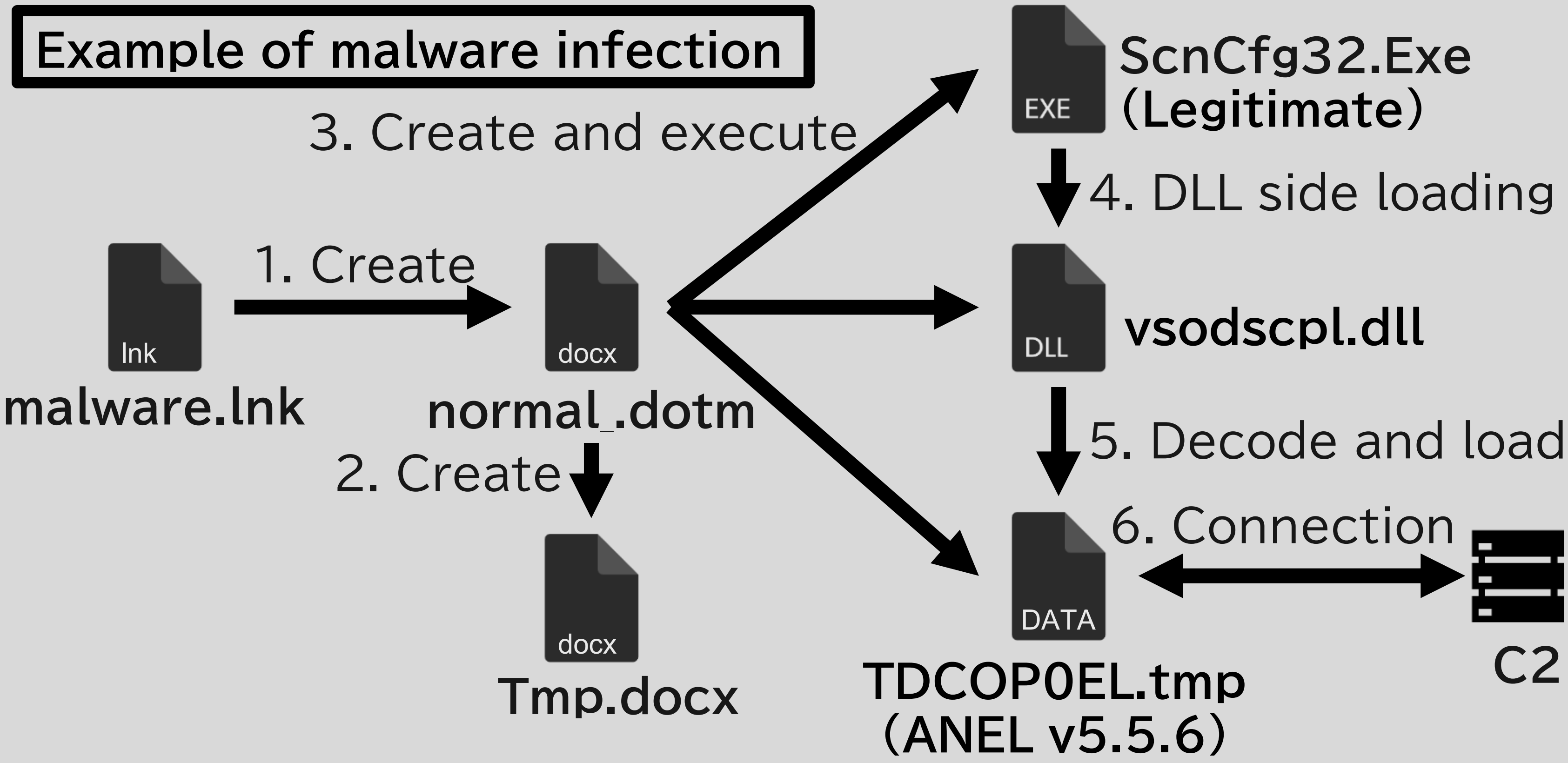
Advanced Attack Behavior Analysis



Advanced Attack Behavior Analysis



How to Write YAMAGoya Custom Rule



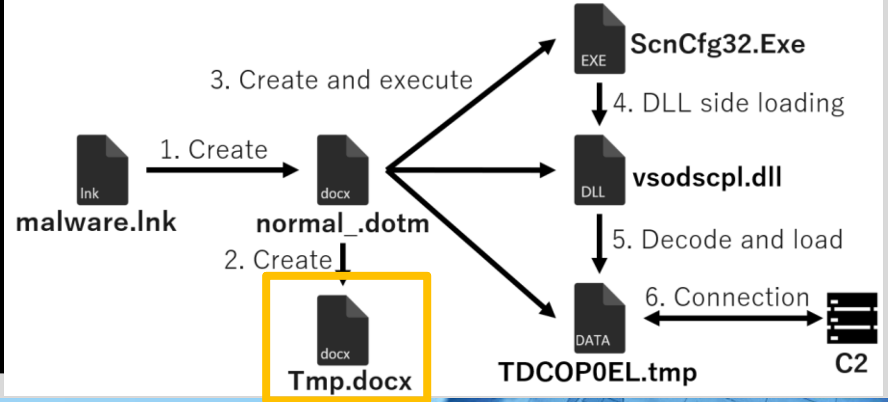
How to Write YAMAGoya Custom Rule

```
rulename: "ANEL"
description: "Detects ANEL from maldoc type"
rules:
  - ruletype: "regex"
    target: "file"
    rule: "Tmp¥¥.docx$"
  - ruletype: "regex"
    target: "process"
    rule: "ScnCfg32¥¥.Exe$"
  - ruletype: "regex"
    target: "dll"
    rule: "vsodscpl¥¥.dll$"
  - ruletype: "regex"
    target: "file"
    rule: "TCDolW0p¥¥.log$"
  - ruletype: "ipv4"
    target: "ipv4"
    rule: "45.32.116.146"
```

How to Write YAMAGoya Custom Rule

```
rulename: "ANEL"
description: "Detects ANEL from maldoc type"
rules:
  - ruletype: "regex"
    target: "file"
    rule: "Tmp¥¥.docx$"
  - ruletype: "regex"
    target: "process"
    rule: "ScnCfg32¥¥.Exe$"
  - ruletype: "regex"
    target: "dll"
    rule: "vsodscpl¥¥.dll$"
  - ruletype: "regex"
    target: "file"
    rule: "TCDolW0p¥¥.log$"
  - ruletype: "ipv4"
    target: "ipv4"
    rule: "45.32.116.146"
```

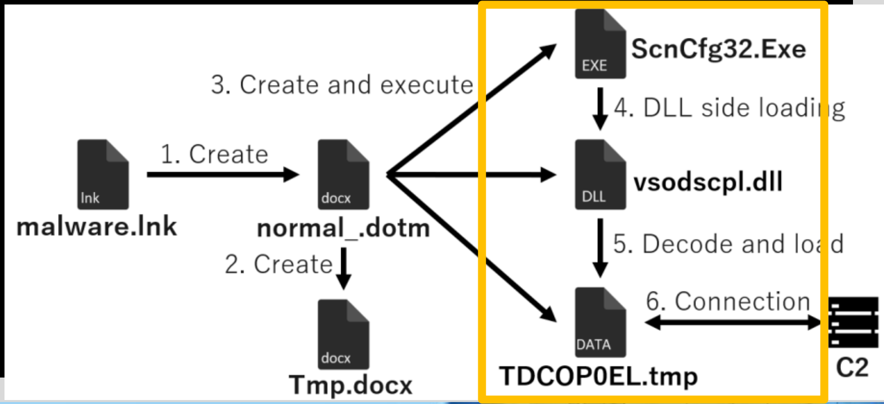
Decoy file



How to Write YAMAGoya Custom Rule

```yaml
rulename: "ANEL"
description: "Detects ANEL from maldoc type"
rules:
  - ruletype: "regex"
    target: "file"
    rule: "Tmp¥¥.docx$"
  - ruletype: "regex"
    target: "process"
    rule: "ScnCfg32¥¥.Exe$"
  - ruletype: "regex"
    target: "dll"
    rule: "vsodscpl¥¥.dll$"
  - ruletype: "regex"
    target: "file"
    rule: "TCDolW0p¥¥.log$"
  - ruletype: "ipv4"
    target: "ipv4"
    rule: "45.32.116.146"
```

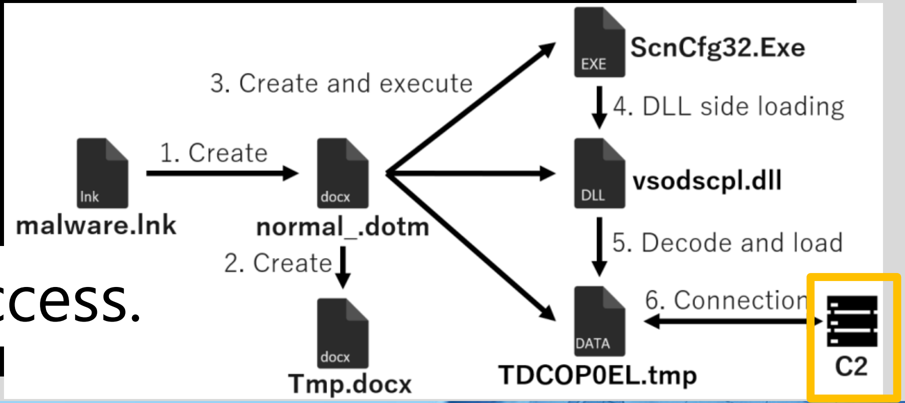
DLL side-loading of malware.



How to Write YAMAGoya Custom Rule

```yaml
rulename: "ANEL"
description: "Detects ANEL from maldoc type"
rules:
  - ruletype: "regex"
    target: "file"
    rule: "Tmp¥¥.docx$"
  - ruletype: "regex"
    target: "process"
    rule: "ScnCfg32¥¥.Exe$"
  - ruletype: "regex"
    target: "dll"
    rule: "vsodscpl¥¥.dll$"
  - ruletype: "regex"
    target: "file"
    rule: "TCDolW0p¥¥.log$"
  - ruletype: "ipv4"
    target: "ipv4"
    rule: "45.32.116.146"
```

C2 access.



How to Write YAMAGoya Custom Rule

```yaml
rulename: "ANEL"
description: "Detects ANEL from maldoc type"
rules:
  - ruletype: "regex"
    target: "file"
    rule: "Tmp¥¥.docx$"
  - ruletype: "regex"
    target: "process"
    rule: "ScnCfg32¥¥.Exe$"
  - ruletype: "regex"
    target: "dll"
    rule: "vsodscpl¥¥.dll$"
  - ruletype: "regex"
    target: "file"
    rule: "TCDolW0p¥¥.log$"
  - ruletype: "ipv4"
    target: "ipv4"
    rule: "45.32.116.146"
```

YAMAGoya detects this malware infection activity if it is checked within the time slot (default 10 seconds).

Demo
- Behavioral detection using Custom YAML rule -

Administrator: Command Prompt

```
C:\YAMAGoya>
```

detect_lsass_dump.yml

```yaml
rulename: "Detect_open_lsass"
description: "Open lsass.exe process to dump memory and extract credentials"
rules:
  - ruletype: "regex"
    target: "open"
    rule: "lsass\\.exe$"
  - ruletype: "regex"
    target: "file"
    rule: "\\.dmp$|\\.dmp$"
```

Spaces: 2 UTF-8 CRLF { } YAML

ENG JA 9:36 PM 7/3/2025

Use Case for Custom Rules (Demo)

Stealing credentials from lsass

```
rulename: "Detect_open_lsass"
description: "Open lsass.exe process to dump
memory and extract credentials"
rules:
  - ruletype: "regex"
    target: "open"
    rule: "lsass¥¥.exe$"
  - ruletype: "regex"
    target: "file"
    rule: "¥¥.dmp$|¥¥.dmp$"
```

Target Activity for Custom Rules

Target Activity			
File	File creation events.	DNS	DNS events.
Delfile	File deletion events.	IPv4	IPv4 network events.
Process	Process events.	IPv6	IPv6 network events.
Open	OpenProcess.	Shell	Shell-related events.
Load	DLL load events.	PowerShell	PowerShell execution events.
Registry	Registry events.	WMI	WMI command execution events.

Use Case for Custom Rules 1

Ransomware Decoy File Detection Technique

```yaml
rulename: "Decoy_file_deleted"
description: "Detecting ransomware using decoy files"
rules:
  - ruletype: "regex"
    target: "delfile"
    rule: "decoy¥¥.docx$"
  - ruletype: "regex"
    target: "delfile"
    rule: "decoy¥¥.png$"
```

Use Case for Custom Rules 2

Detecting WMI Activity used by Malware

```
rulename: "WMI_command_activity"
description: "Detects WMI commands using attacker"
rules:
  - ruletype: "regex"
    target: "wmi"
    rule: "SELECT ¥¥* FROM Win32_NetworkAdapterConfiguration"
  - ruletype: "regex"
    target: "wmi"
    rule: "SELECT ¥¥* FROM Win32_OperatingSystem"
  - ruletype: "regex"
    target: "wmi"
    rule: "SELECT ¥¥* FROM Win32_ComputerSystem"
```

Use Case for Custom Rules 3

Detection of PowerShell Activity

```yaml
rulename: "PowerShell_command_activity"
description: "Detects Invoke-SharpLoader.ps1"
rules:
  - ruletype: "regex"
    target: "powershell"
    rule: "MemoryStream"
  - ruletype: "regex"
    target: "powershell"
    rule: "GZipStream"
  - ruletype: "regex"
    target: "powershell"
    rule: "EntryPoint.Invoke"
```

JPCERTCC / YAMAGoya-Private

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

main

YAMAGoya-Private / README.md

Go to

shu-tom Upload ✓

Preview | Code | Blame | 499 lines (386 loc) · 21.2 KB

Raw

YAMAGoya

Concept

YAMAGoya (Yet Another Memory Analyzer for malware detection and Guarding Operations with YARA and SIGMA) is a C# application that leverages [Event Tracing for Windows (ETW)](#) to capture real-time system events. It applies detection rules written in YAML format (for custom correlation logic) and can also parse **SIGMA** rules for standardized threat detection. In addition, it supports in-memory scanning using **YARA** to detect fileless or stealth malware.

The tool run: **https://github.com/JPCERTCC/YAMAGoya/** itures.

Table of Contents

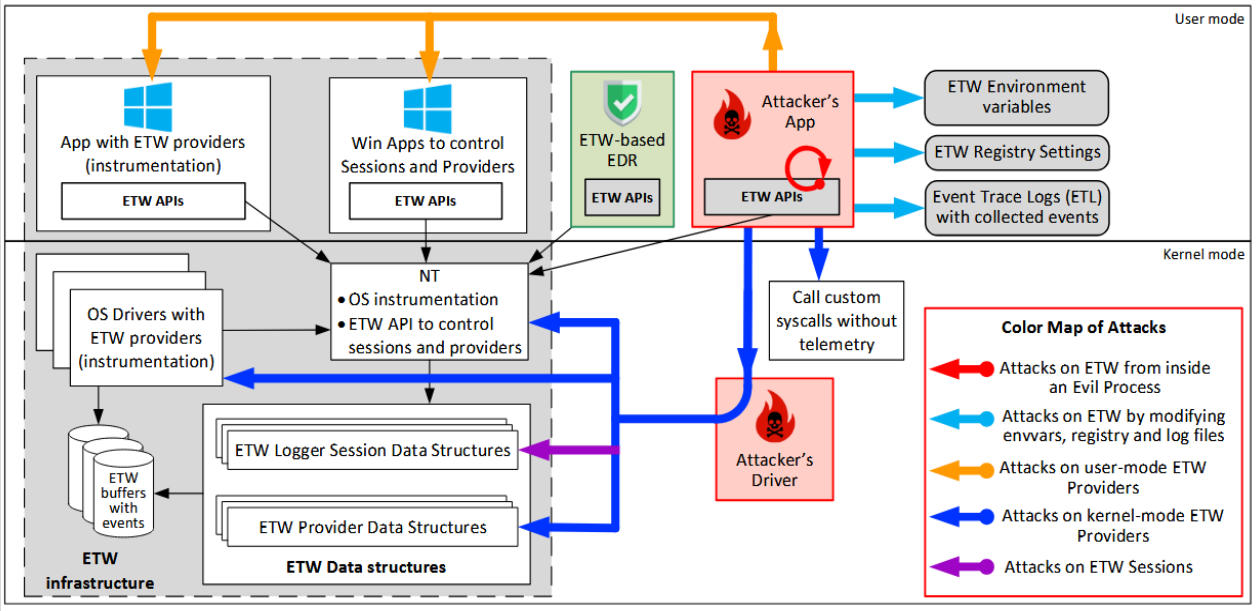
Weakness of YAMAGoya

YAMAGoya might not detect properly malicious behavior when an attacker uses **ETW bypass** techniques

Campaigns using ETW bypass

https://blogs.jpcert.or.jp/en/2025/07/ivanti_cs.html
https://www.sonicwall.com/blog/remcos-rat-targets-europe-new-amsi-and-etw-evasion-tactics-uncovered
https://www.trendmicro.com/vinfo/us/security/news/cybercrime-and-digital-threats/earth-baku-returns

Attack surfaces of ETW



https://i.blackhat.com/EU-21/Wednesday/EU-21-Teodorescu-Veni-No-Vidi-No-Vici-Attacks-On-ETW-Blind-EDRs.pdf

Detectable ETW bypass by YAMAGoya

- ✓ Config modification via PowerShell
- ✓ Registry modification
- ✓ ETL file deletion

```
rulename  “KillETW"
description  “Detects KillETW.ps1"
rules
    ruletype  "regex"
    target: "powershell"
    rule: "System.Diagnostics.Eventing.EventProvider"
  - ruletype: "regex"
    target: "powershell"
    rule:
"System.Management.Automation.Tracing.PSEtwLogProvider"
  - ruletype: "regex"
    target: "powershell"
    rule: "SetValue"
```

```
C:\YAMAGoya>YAMAGoya.exe -s -d rules -a

C:\YAMAGoya>Running in command-line mode...
[INFO] Using default session name: YAMAGoya
Starting ETW session...
ETW session started. Press Ctrl+C to stop.
[INFO] Loading YAML rule files...
[INFO] Loading detection rules from C:\YAMAGoya\rules\detect_invoke-sharploader.yml
[INFO] Loading detection rules from C:\YAMAGoya\rules\detect_killetw.yml
[INFO] Loading detection rules from C:\YAMAGoya\rules\detect_lsass_dump.yml
[INFO] Loading detection rules from C:\YAMAGoya\rules\detect_malware_sample.yml
[INFO] Loading detection rules from C:\YAMAGoya\rules\detect_ransomeware_using_decoy.yml
[INFO] Loading detection rules from C:\YAMAGoya\rules\detect_wmi_activity.yml
[INFO] Successfully loaded 6 rules. (Skipped 0 files with errors)
[INFO] Loaded default rule count: 6
[INFO] Press Ctrl+C to stop.
[INFO] Starting ETW detection (check interval: 10s)...
[INFO] 2025/07/25 11:51:39 DETECTED: KillETW - Detects KillETW.ps1
```

Undetectable ETW bypass by YAMAGoya

- Patching user mode ETW provider
- Attack on kernel mode ETW provider
- Attack on ETW session

```
void DisableETW(void) {
    DWORD oldprotect = 0;

    unsigned char sEtwEventWrite[] = { 'E','t','w','E','v','e','n','t','W','r','i','t','e', 0x0 };

    // change the protection so we can write to it
    void* pEventWrite = GetProcAddress(GetModuleHandle(L"ntdll.dll"), (LPCSTR)sEtwEventWrite);

    VirtualProtect_p(pEventWrite, 4096, PAGE_EXECUTE_READWRITE, &oldprotect);

#ifdef _WIN64
    // the return from this function will be 0
    memcpy(pEventWrite, "\x48\x33\xc0\xc3", 4);                // xor rax, rax; ret
#else
    memcpy(pEventWrite, "\x33\xc0\xc2\x14\x00", 5);            // xor eax, eax; ret 14
#endif

    VirtualProtect_p(pEventWrite, 4096, oldprotect, &oldprotect);
    FlushInstructionCache(GetCurrentProcess(), pEventWrite, 4096);
}
```

https://github.com/ASkyeye/FilelessRemotePE/blob/main/RemotePE/RemotePE.cpp

Demo
- ETW bypass detection -

Administrator: コマンド プロンプト

```
C:\YAMAGoya>
```

Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

Detect_bypass_etw.yml

C: > YAMAGoya > rules > Detect_bypass_etw.yml

```yaml
rulename: "BypassETW"
description: "Detects 8KuV.ps1 in the campaign of REMCOS RAT ta
rules:
  - ruletype: "regex"
    target: "powershell"
    rule: "0x45,0x74,0x77,0x45,0x76,0x65,0x6E,0x74,0x57,0x72,0x
  - ruletype: "regex"
    target: "powershell"
    rule: "0xC3"
  - ruletype: "regex"
    target: "powershell"
    rule: "0xb8,0xff,0x55"
  - ruletype: "regex"
    target: "powershell"
    rule: "[System.Text.Encoding]::ASCII.GetString"

```

Restricted Mode ⊗ 0 ⚠ 0 Ln 1, Col 22 Spaces: 2 UTF-8 LF YAML

33°C 晴れ 20:09 2025/07/27

Custom Rules (Demo2)

Ransomware Decoy File Detection Technique

```
rulename  “BypassETW"
description  "Detects 8KuV.ps1 in the campaign of REMCOS RAT targets Europe"
rules
  - ruletype: "regex"
    target: "powershell"
    rule: "0x45,0x74,0x77,0x45,0x76,0x65,0x6E,0x74,0x57,0x72,0x69,0x74,0x65"
    ruletype  "regex"
    target: "powershell"
    rule: "0xC3"
  - ruletype  "regex"
    target: "powershell"
    rule: "0xb8,0xff,0x55"
  - ruletype  "regex"
    target: "powershell"
    rule: "[System.Text.Encoding]::ASCII.GetString"
```

“EtwEventWrite”

```
$etwFuncName = "EtwEventWrite"
$etwAddr = Get-SystemProcAddress ("ntdll.dll") $etwFuncName
if ($etwAddr -eq $null) {
    Throw "[!] Erro ao obter o endereço de $etwFuncName"    #Error getting address of $etwFuncName
}

if (-not $memProtectDelegate.Invoke($etwAddr, 1, $PAGE_EXECUTE_WRITECOPY, [ref]$origProt)) {
    Throw "[!] Erro ao alterar a proteção de memória em $etwFuncName"    #Error changing memory protection on $etwFuncName
}

try {
    if ($PtrSize -eq 8) {
        $marshal::WriteByte($etwAddr, 0xC3)
    }
    else {
        $etwPatch = [byte[]](0xb8,0xff,0x55)
        $marshal::Copy($etwPatch, 0, [IntPtr]$etwAddr, 3)
    }
```

Patching EtmEventWrite by
Either Ret[0xC3]
Or Mov eax,00000NAN[0xb8,0xff,0x55]

https://www.sonicwall.com/blog/remcos-rat-targets-europe-new-amsi-and-etw-evasion-tactics-uncovered

FAQ

Q1: Can YAMAGoya replace traditional antivirus software?
A: **No.** YAMAGoya is designed to complement antivirus software by focusing on behavioral and memory-based detection, especially for fileless malware and advanced attacks.

Q2: Can YAMAGoya run continuously in the background?
A: **Yes.** YAMAGoya supports system tray integration and can run quietly in the background while actively monitoring for threats using the configured rules.

Q3: Can YAMAGoya be integrated into an existing SIEM platform?
A: **Yes.** Although YAMAGoya is a standalone tool, it outputs logs in text and Windows Event Log formats. These can be forwarded to SIEMs like Splunk using standard log collectors or agents.

FAQ

Q4: Are there any known limitations regarding ETW Bypass?

A: **Yes.** YAMAGoya currently does not implement specific countermeasures against ETW bypass techniques. This means advanced attackers may attempt to disable or tamper with ETW to avoid detection. We recommend using YAMAGoya in combination with other monitoring solutions for a more robust defense.

Thank you!

@jpcert_en
@jpcert_ac

ir-info@jpcert.or.jp
PGP https://www.jpcert.or.jp/english/pgp/